UNIVERSIDAD TECNOLOGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Programación I

Trabajo Práctico Integrador

Sistema de Control de Inventario en Python

Informe de diseño y fundamentación

Profesor: Neyén Bianchi

Alumno: Joaquín Dalmiro Lallana

DNI: 39.386.391

1. Introducción **2**
2. Conceptos aplicados **3**
2.1 Listas 3
2.2 Diccionarios 3
2.3 Funciones 5
2.4 Condicionales 6
2.5 Bucles 7
2.6 Manejo de errores: try, except y raise 8
3. Diagrama de flujo **10**
3.1 Menú principal 10
3.2 Carga de herramientas y validación de datos 12
3.3 Actualización de stock 14
4. Lenguaje, entorno y recursos utilizados **16**
5. Conclusiones **18**

1. Introducción

El trabajo consiste en un programa de consola escrito en Python que lleva el control de inventario de una ferretería. Desde un menú de siete opciones se pueden cargar varias herramientas con su stock inicial, listar el inventario, consultar el stock de una herramienta, ver las herramientas agotadas, dar de alta un producto, registrar ventas e ingresos, y salir del sistema.

El programa está dividido en archivos .py propios, organizados en tres paquetes (carpetas que agrupan archivos relacionados) más el archivo principal. Todo el código está en la carpeta sistema_de_inventario, que también incluye un archivo README.md con la descripción del proyecto y las instrucciones de uso. main.py es el punto de entrada: contiene main(), con el bucle del menú, y la llamada final a main(). Para iniciar el programa se ejecuta python3 main.py desde la carpeta sistema_de_inventario. El paquete interfaz contiene menu.py, con mostrar_menu(), que imprime las opciones, y pedir_opcion(), que lee la opción elegida. El paquete operaciones agrupa las funciones del menú según lo que hacen con el inventario: carga.py (opciones 1 y 5, que agregan herramientas), consultas.py (opciones 2, 3 y 4, que solo lo leen) y movimientos.py (opción 6, que modifica el stock). El paquete auxiliares contiene las funciones reutilizables: validaciones.py (pedir_entero, validar_nombre y validar_nombre_nuevo) y busqueda.py (buscar_herramienta).

Elegí esta división para que cada archivo tenga una responsabilidad clara y se encuentre rápido dónde está cada parte: si hay un error en una venta, se busca en operaciones/movimientos.py; si falla una validación, en auxiliares/validaciones.py. Cada archivo trae las funciones que necesita de otro con from paquete.modulo import funcion, por ejemplo from auxiliares.validaciones import pedir_entero. Esos import son solo de archivos propios: el programa no usa ninguna librería externa. A nivel de módulo, cada archivo tiene únicamente sus import y sus definiciones de funciones, y main.py además tiene la llamada final a main(). No hay variables globales: inventario se crea dentro de main() y se pasa por parámetro a las funciones de los otros archivos.

En las secciones siguientes explico cada concepto de la materia a partir de la parte del programa donde lo apliqué. Cuando cito un número de línea, indico el archivo al que corresponde, en la versión que se entrega.

2. Conceptos aplicados

2.1 Listas

El inventario se guarda en una única lista llamada inventario, que se crea vacía dentro de main() (main.py, línea 13, inventario = []). No es una variable global: main() se la pasa por parámetro a cada función del menú, por ejemplo cargar_herramientas(inventario).

Pasarla por parámetro funciona porque una lista es un objeto mutable. Cuando cargar_herramientas o alta_producto ejecutan inventario.append(...) (operaciones/carga.py, líneas 31 y 42), modifican la misma lista que creó main(), no una copia. Por eso, al volver al menú, las herramientas cargadas siguen ahí, y una segunda ejecución de la opción 1 agrega a lo que ya estaba.

Sobre la lista uso tres operaciones:

- append para agregar una herramienta nueva al final, solo cuando el nombre y el stock ya fueron validados.
- len(inventario) == 0 para detectar el inventario vacío en las opciones 2, 3, 4 y 6, antes de pedir cualquier dato.
- Recorridos con for producto in inventario para listar, buscar, contar agotadas y detectar duplicados.

La consigna no permite otras colecciones, así que no guardo resultados intermedios en listas auxiliares. Por ejemplo, el reporte de agotados no arma una lista de herramientas con stock cero: recorre inventario y las muestra directamente (ver 2.5).

2.2 Diccionarios

Cada elemento de inventario es un diccionario con exactamente dos claves: 'herramienta', con el nombre como texto, y 'cantidad', con el stock como número entero. Por ejemplo: {'herramienta': 'Martillo', 'cantidad': 10}.

El diccionario de una herramienta nueva se arma directamente dentro del append, sin guardarlo antes en una variable: inventario.append({'herramienta': nombre, 'cantidad': stock}). Así no

existe ningún diccionario fuera de la lista.

Para leer los datos accedo por clave, como en mostrar_inventario, que imprime producto['herramienta'] y producto['cantidad'] (operaciones/consultas.py, línea 15). Para modificar el stock en la opción 6 aprovecho que buscar_herramienta devuelve el mismo diccionario que está dentro de la lista, no una copia. Entonces, cuando actualizar_stock asigna un nuevo valor a producto['cantidad'] (operaciones/movimientos.py, líneas 30 y 33), el cambio queda guardado en inventario sin tener que buscar la posición de la herramienta.

El nombre se guarda con strip(), respetando las mayúsculas que escribió el usuario: si se carga " Llave Inglesa ", queda guardado "Llave Inglesa". Para comparar nombres, en cambio, buscar_herramienta pasa los dos nombres a minúsculas con lower() y le saca los espacios con strip() al nombre buscado (el guardado ya no los tiene), de modo que "Martillo", "martillo" y " MARTILLO " se consideran la misma herramienta.

2.3 Funciones

Dividí el programa en funciones con una tarea cada una. Las opciones 1 a 6 tienen su propia función, y la opción 7 se resuelve dentro de main(). Las validaciones que se repiten en varias opciones las separé en funciones auxiliares, para escribir cada regla una sola vez. Además, agrupé las funciones en archivos y paquetes según su responsabilidad, como se describe en la Introducción.

Función	Tarea
main()	Crea inventario, muestra el menú en bucle (lo imprime mostrar_menu() y lee la opción con pedir_opcion()), llama a la función de cada opción y atrapa los ValueError que llegan desde las opciones.
cargar_herramientas(inventario)	Opción 1. Carga varias herramientas y reintenta cada dato inválido dentro de la función.
mostrar_inventario(inventario)	Opción 2. Lista todas las herramientas con su stock.
consultar_stock(inventario)	Opción 3. Busca una herramienta por nombre y muestra su stock.
reporte_agotados(inventario)	Opción 4. Muestra solo las herramientas con stock cero.
alta_producto(inventario)	Opción 5. Agrega una herramienta; si un dato es inválido, el error sale de la función.
actualizar_stock(inventario)	Opción 6. Registra una venta o un ingreso, en el orden de pasos que pide la consigna.
pedir_entero(mensaje, minimo)	Pide un entero con int(input(...)); lanza ValueError en español si no es entero o es menor que el mínimo.
validar_nombre(nombre)	Lanza ValueError si el nombre está vacío o tiene solo espacios y, si no, lo devuelve sin espacios en los extremos.
validar_nombre_nuevo(inventario, nombre)	Valida el nombre con validar_nombre y lanza ValueError si la herramienta ya está en el inventario (duplicado). Se usa en las opciones 1 y 5.
buscar_herramienta(inventario, nombre)	Devuelve el diccionario de la herramienta encontrada, o None si no existe.

El caso más claro del uso de parámetros es pedir_entero. La misma función pide y valida tres datos con reglas distintas porque recibe el mensaje y el mínimo como parámetros: pedir_entero(..., 1) para la cantidad de herramientas a cargar, pedir_entero(..., 0) para el stock inicial, donde el 0 es válido, y pedir_entero(..., 1) para las unidades del movimiento de una

venta o un ingreso. Así la conversión con int() y la regla del mínimo están escritas una sola vez (auxiliares/validaciones.py, líneas 6 a 16). Con el mismo criterio, validar_nombre_nuevo reúne en un solo lugar la validación del nombre de una herramienta nueva (vacío y duplicado), que usan las opciones 1 y 5.

También uso return de dos maneras. En las auxiliares, return devuelve un resultado, como el nombre limpio de validar_nombre o el diccionario de buscar_herramienta. En las funciones de las opciones, un return sin valor sirve para terminar la función antes de tiempo, por ejemplo cuando el inventario está vacío, y volver al menú sin pedir datos.

2.4 Condicionales

El menú de main() es una cadena if / elif / else (main.py, líneas 19 a 35). Cada número del 1 al 6 llama a su función, el 7 muestra el mensaje de salida y termina el bucle, y cualquier otro número entero cae en el else, que lanza raise ValueError("Opción fuera de rango..."). Uso else y no elif opcion != 7 porque el 7 ya tiene su propia rama, de modo que al else solo llegan los números inválidos.

En la opción 6 el tipo de movimiento se compara como texto, sin convertirlo con int(), con la condición movimiento != '1' and movimiento != '2' (operaciones/movimientos.py, línea 21). Va and y no or: con or la condición sería verdadera para cualquier valor, porque ningún texto puede ser igual a '1' y a '2' al mismo tiempo. Si el tipo no es válido, la función muestra un mensaje y termina con return, sin lanzar una excepción, como pide la consigna para ese caso.

Otros condicionales importantes son:

- El control de inventario vacío con if len(inventario) == 0 al comienzo de las opciones 2, 3, 4 y 6.
- La verificación de stock suficiente antes de una venta: si producto['cantidad'] < unidades, se lanza un ValueError y el stock no se modifica (operaciones/movimientos.py, líneas 28 y 29).
- El if producto == None de las opciones 3 y 6, que distingue entre una herramienta que no existe (mensaje y vuelta al menú) y una que sí existe (se muestra su stock o se sigue con el movimiento).
- El if contador_agotados == 0 de reporte_agotados, que elige entre el mensaje "todas tienen stock" y el listado de agotadas. Ese mensaje es distinto del de inventario vacío.

2.5 Bucles

Bucle del menú. El menú persistente es un while True dentro de main() (main.py, línea 15). En cada vuelta se imprime el menú con mostrar_menu(), se pide la opción con pedir_opcion() y se ejecuta la función correspondiente. La única salida es la opción 7, que muestra "Saliendo del sistema..." y ejecuta break. Elegí while True con break, en lugar de while opcion != 7, porque así la opción 7 tiene su rama explícita en el if, igual que las demás, y el else queda solo para las opciones inválidas. Las dos formas son equivalentes y break es una de las estructuras permitidas.

Bucles de reintento. En cargar_herramientas uso tres bucles while True, cada uno con un try / except y un break al final del try: uno para la cantidad de herramientas, otro para el nombre y otro para el stock. Si el dato es válido se llega al break y se sale del bucle. Si no, el except muestra el error y el while vuelve a pedir ese mismo dato. Como el bucle del stock está separado del bucle del nombre, un stock inválido vuelve a pedir solo el stock y el nombre ya validado se conserva.

Bucle con cantidad conocida. La carga de varias herramientas usa for i in range(cantidad_herramientas), porque la cantidad de vueltas se conoce después de pedir la cantidad de herramientas a cargar. Uso i + 1 para mostrar "herramienta 1", "herramienta 2", etc.

Recorridos de búsqueda. buscar_herramienta recorre inventario con for producto in inventario y, cuando encuentra la herramienta, termina con return producto dentro del for. Es la única función que busca: la usan consultar_stock, actualizar_stock y validar_nombre_nuevo (para detectar duplicados). El caso "no la encontré" se resuelve recién después del for (return None), porque solo se sabe que no está cuando se recorrió toda la lista. Si ese caso estuviera dentro del for, la búsqueda terminaría en la primera herramienta que no coincide.

Contador. reporte_agotados recorre inventario dos veces. En la primera cuenta las herramientas con stock cero en la variable contador_agotados. En la segunda, solo si contador_agotados es mayor que cero, las muestra. Lo hice así porque el título del listado tiene que aparecer una sola vez y solo cuando hay agotadas, y para eso necesito saberlo antes de imprimir. El contador es una variable simple, así que no hace falta una lista auxiliar.

2.6 Manejo de errores: try, except y raise

El programa usa una sola excepción, ValueError, que es la que trae Python. Aparece de dos formas:

- La lanza Python por sí mismo cuando int() no puede convertir un texto, por ejemplo int('abc') o int('3.5').
- La lanzo yo con raise ValueError("mensaje") cuando un dato se puede convertir pero rompe una regla del negocio: nombre vacío o duplicado, stock negativo, cantidad de herramientas a cargar o unidades menores que 1, venta mayor que el stock y opción del menú fuera de rango.

Las funciones auxiliares de validación lanzan el error pero no lo resuelven. Así, cada opción decide qué hacer con él, y uso dos estrategias distintas según lo que pide la consigna.

Opción 1: el error se atrapa dentro de la función. En cargar_herramientas, cada dato se pide dentro de un while True con try / except ValueError as e. El except muestra el motivo con print(f"Error: {e}") y el bucle vuelve a pedir el dato. El error nunca llega a main(), y las herramientas ya cargadas en esa misma ejecución se conservan.

Opciones 3, 5 y 6: el error sale de la función. consultar_stock, alta_producto y actualizar_stock no tienen ningún try. Cuando una validación falla, la función se corta en esa línea y el ValueError sube hasta el except ValueError as e de main() (main.py, línea 36), que muestra el motivo y vuelve al menú. El error viaja igual aunque la función esté en otro archivo: una excepción que no se captura sube por la cadena de llamadas, sin importar en qué módulo esté definida cada función. Este diseño garantiza que no queden datos a medias: en alta_producto, el append va después de todas las validaciones, así que solo se ejecuta si el nombre y el stock pasaron todas las validaciones. En actualizar_stock, el stock se modifica recién al final, después de validar el nombre, que la herramienta esté en el inventario, el tipo de movimiento, las unidades y el stock suficiente.

El mismo except de main() atrapa también los errores del propio menú: una opción que no es un número, como abc, y una opción entera fuera de rango, como 9. En el primer caso, el mensaje que trae el ValueError de int() está en inglés (invalid literal for int()), así que pedir_opcion() (interfaz/menu.py) lo captura con try / except y lo vuelve a lanzar con raise ValueError y un mensaje en español que pide un número entero del 1 al 7. pedir_entero hace lo

mismo con la cantidad, el stock y las unidades. En ambos casos el error se relanza porque, con un simple print, ni la opción 1 podría reintentar ni main() se enteraría.

Hay tres casos que no son errores sino situaciones informativas, y los resolví con una condición y un mensaje, sin excepción: el inventario vacío, una herramienta que no existe y un tipo de movimiento inválido. En esos casos la función muestra el mensaje y termina con return, sin modificar nada.

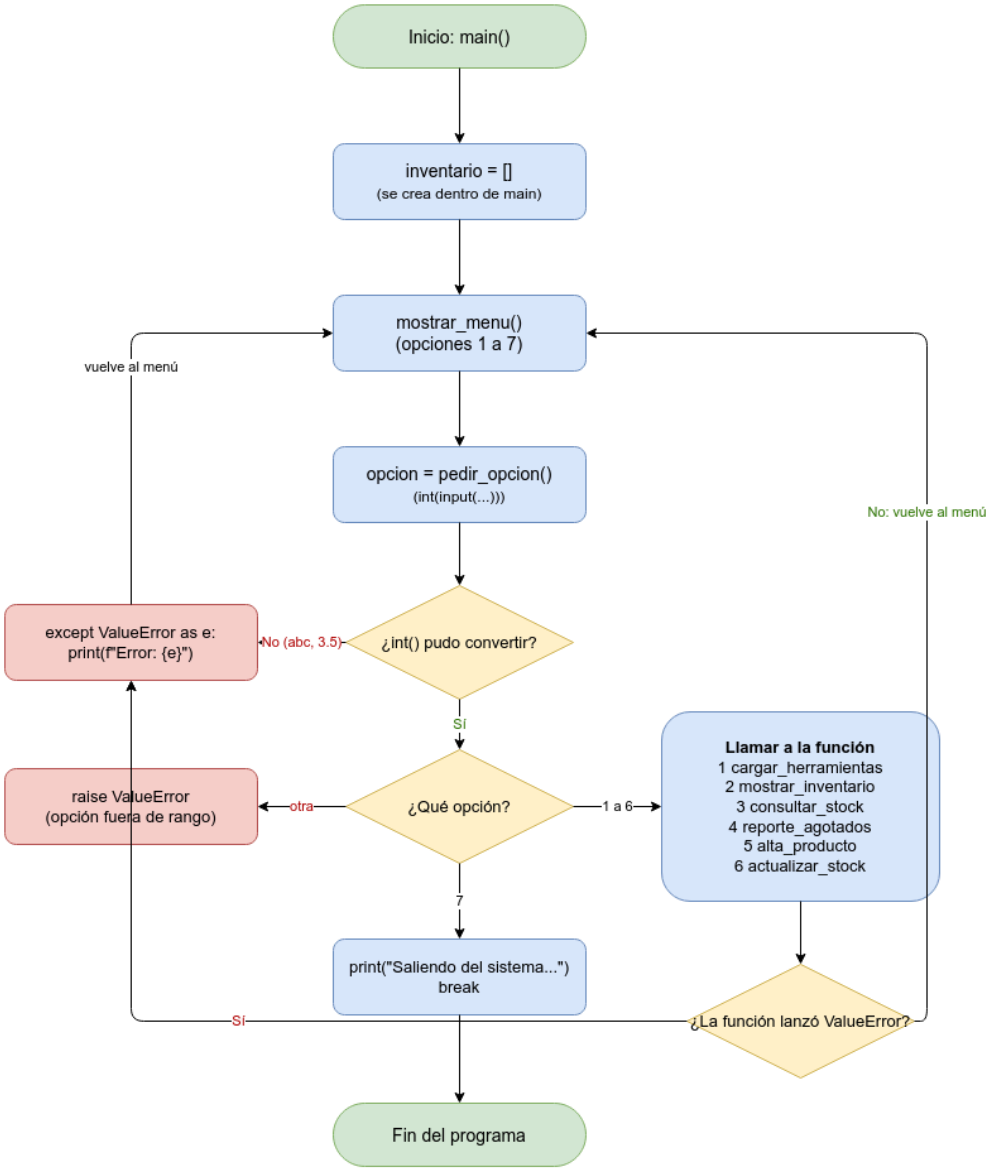
Situación	Mecanismo	Dónde se resuelve
Nombre vacío, solo espacios o duplicado	raise ValueError	validar_nombre y validar_nombre_nuevo. Opción 1: reintenta. Opciones 3, 5 y 6: main().
Dato no convertible a entero (abc, 3.5)	ValueError de int()	Relanzado en español por pedir_entero y pedir_opcion. Opción 1: reintenta. Opciones 5, 6 y menú: main().
Stock inicial negativo (el 0 es válido)	raise ValueError	pedir_entero(..., 0)
Cantidad a cargar o unidades menores que 1	raise ValueError	pedir_entero(..., 1)
Venta mayor que el stock	raise ValueError	actualizar_stock, antes de modificar el stock
Opción del menú fuera de 1 a 7	raise ValueError	else del menú en main()
Inventario vacío o herramienta inexistente	Condición y mensaje	Opciones 2, 3, 4 y 6, con return
Tipo de movimiento inválido	Condición y mensaje	actualizar_stock, con return

3. Diagrama de flujo

Hice el diagrama en draw.io a partir de mi código. Lo dividí en tres figuras para que cada recorrido se lea con claridad. En las tres uso los mismos colores: azul para los procesos, amarillo para las decisiones, verde para el inicio, el fin y los cambios en el inventario, rojo para los errores ValueError y naranja para los mensajes informativos que no usan excepciones.

3.1 Menú principal

Figura 1. Menú principal: función main()

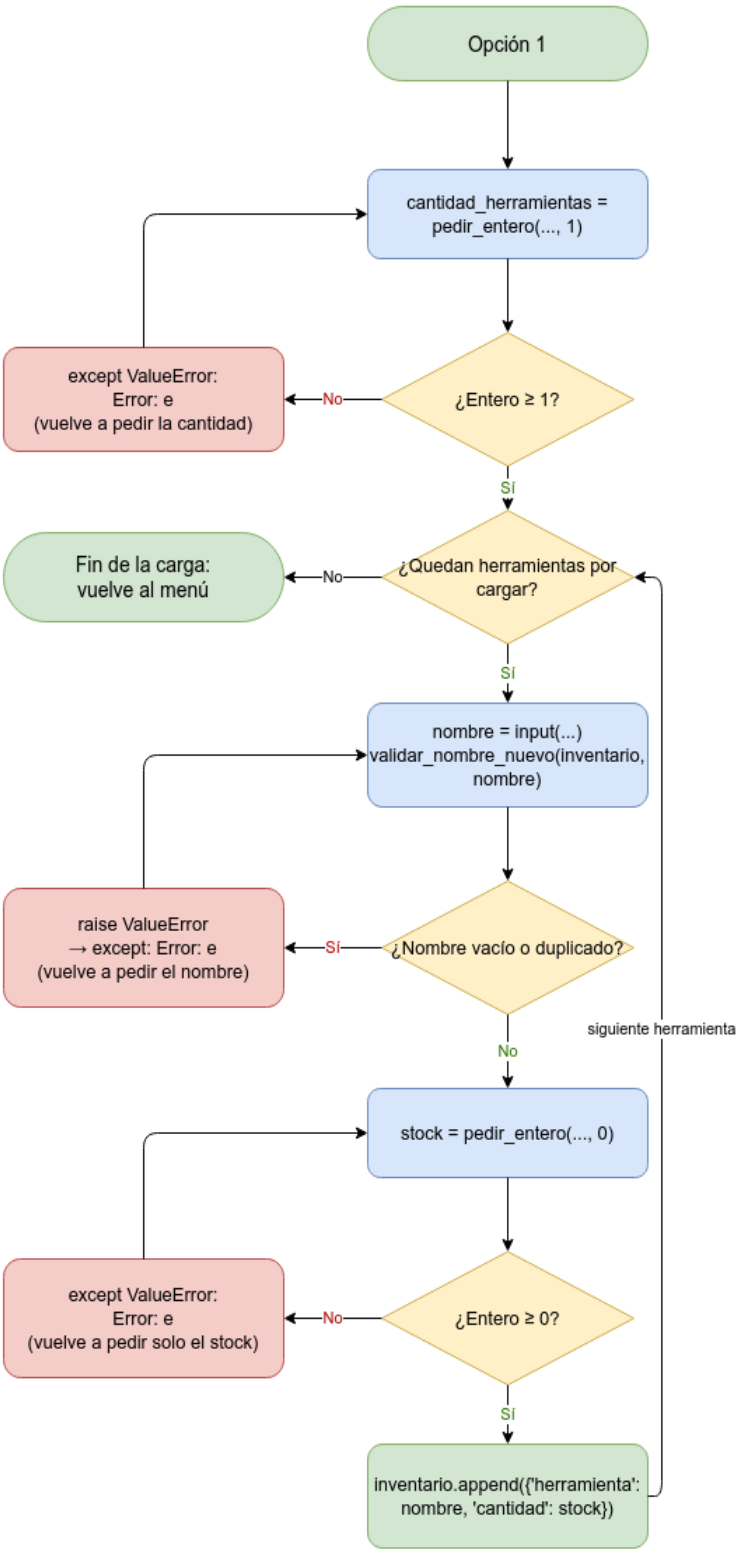


Azul: proceso · Amarillo: decisión · Verde: inicio y fin · Rojo: error ValueError

La Figura 1 muestra main(). Después de crear inventario, el programa entra al bucle del menú. Si la opción no se puede convertir a entero, o si es un entero fuera de rango, el error llega al except y se vuelve a mostrar el menú. Las opciones 1 a 6 llaman a su función; si esa función deja salir un ValueError, también lo atrapa el except de main(). La opción 7 ejecuta break y termina el programa.

3.2 Carga de herramientas y validación de datos

Figura 2. Opción 1: carga múltiple con validación (cargar_herramientas)

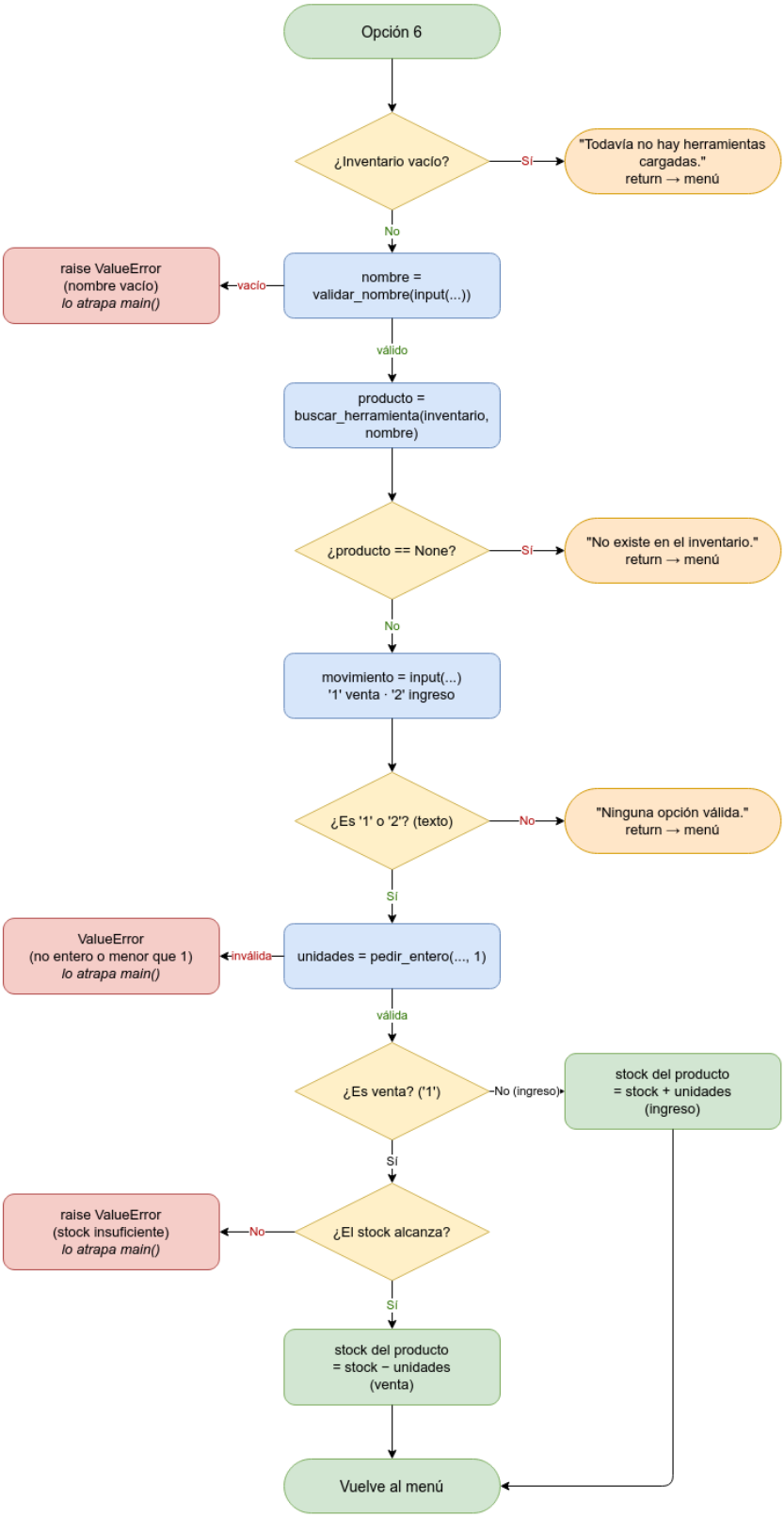


Azul: proceso · Amarillo: decisión · Verde: inicio, fin y alta en el inventario · Rojo: error capturado dentro de la función

La Figura 2 muestra la opción 1, que es el recorrido con más validaciones. Tiene tres ciclos de reintento, uno por cada dato: la cantidad de herramientas, que debe ser un entero mayor o igual a 1; el nombre, que no puede estar vacío ni repetido; y el stock inicial, que debe ser un entero mayor o igual a 0. Cada error vuelve a pedir solo el dato que falló. La herramienta se agrega con append únicamente cuando el nombre y el stock son válidos, y el for continúa con la siguiente hasta completar la cantidad pedida.

3.3 Actualización de stock

Figura 3. Opción 6: actualización de stock (actualizar_stock)



Azul: proceso · Amarillo: decisión · Verde: inicio, fin y cambio de stock · Naranja: mensaje informativo, sin excepción · Rojo: ValueError que llega a main() (el stock no cambia)

La Figura 3 muestra la opción 6 en el orden que exige la consigna: inventario vacío, nombre, herramienta en el inventario, tipo de movimiento, unidades y, si es una venta, stock suficiente. Los casos informativos (en naranja) terminan la función con return. Los errores (en rojo) son ValueError que llegan a main(). En ninguno de esos caminos se modifica el stock: el stock del producto cambia solo en los dos bloques verdes del final, uno para la venta y otro para el ingreso.

4. Lenguaje, entorno y recursos utilizados

Python 3.12.3. Es el lenguaje que pide la materia. Es un lenguaje interpretado, con una sintaxis clara y estructuras como listas y diccionarios incluidas sin importar librerías, que son justo las que exige el trabajo. Usé la versión 3.12.3, que es la que viene instalada en mi sistema operativo, y no necesité ninguna librería externa. El programa se ejecuta con python3 main.py, desde la carpeta sistema_de_inventario.

Visual Studio Code 1.144.0. Es el editor de código que usé para escribir el programa. Lo elegí porque resalta la sintaxis de Python, marca errores de sangría mientras escribo y tiene una terminal integrada, así que puedo editar y probar el programa en la misma ventana. También es el entorno que pide la consigna para mostrar el código en el video.

Linux Mint 22.3. Es el sistema operativo de mi computadora, una distribución de Linux basada en Ubuntu, con kernel 7.0.0. Trae Python 3 instalado de fábrica, lo que me permitió empezar a programar sin instalar nada más.

Terminal Bash 5.2.21. Es el intérprete de comandos de Linux. Lo usé para ejecutar el programa y para correr las pruebas automáticas. Elegí Bash porque es la terminal que trae el sistema y porque permite pasarle al programa varias entradas seguidas, lo que hace posible repetir las pruebas sin escribirlas a mano cada vez.

Claude Code 2.1.316. Es un asistente de programación de Anthropic que funciona en la terminal y en VS Code. Lo usé como tutor: me explicó conceptos como raise, el alcance de return y la diferencia entre métodos y funciones con ejemplos ajenos al inventario. También me ayudó a encontrar errores, a auditar el funcionamiento del programa contra las reglas y las validaciones de la consigna, y a correr pruebas de ejecución automáticas para confirmar que cada opción funciona correctamente.

draw.io (diagrams.net). Es una herramienta gratuita para hacer diagramas, que se usa desde el navegador. La elegí para el diagrama de flujo porque tiene las formas estándar (procesos, decisiones, inicio y fin) y permite exportar el resultado como imagen para insertarlo en este informe.

Google Docs. Es el procesador de texto en línea de Google, donde redacté este informe. Lo elegí porque guarda los cambios automáticamente y permite exportar el documento a PDF, que es el formato de entrega.

5. Conclusiones

Una de las cosas que más me dejó este trabajo es que la mayor parte del esfuerzo no está en hacer la operación en sí, sino en anticipar lo que puede salir mal. Registrar una venta es una resta; lo difícil es prever que alguien escriba una letra donde va un número, deje el nombre vacío o intente vender más de lo que hay. La idea que terminó ordenando todo el programa fue simple: primero validar y recién después modificar el inventario, para que un error nunca deje los datos a medias.

También aprendí que no alcanza con detectar un error: hay que decidir dónde resolverlo y cómo comunicarlo. En una carga de varias herramientas conviene volver a pedir solo el dato que falló, para no obligar al usuario a empezar de nuevo; en cambio, en un alta o en una venta es más claro cancelar la operación y volver al menú sin tocar nada. Y los mensajes tienen que hablarle a quien usa el programa: por eso reemplacé los avisos en inglés que muestra Python por mensajes claros en español.

Otra enseñanza fue el valor de ordenar el código. Dividir el programa en funciones con una sola tarea, y después agruparlas en carpetas según lo que hacen, hizo más fácil encontrar cada parte, corregirla y explicarla. Algo parecido pasó con las restricciones de la consigna: trabajar con una sola lista, sin variables globales y con un conjunto reducido de herramientas me obligó a resolver cada problema con lo básico, y eso me ayudó a entender de verdad cómo funciona cada estructura.

Por último, comprobé que probar el programa no es un paso final sino parte del trabajo: repetir los mismos casos después de cada cambio me permitió detectar errores que a simple vista no se notaban. Como mejora a futuro, el programa podría guardar el inventario en un archivo para no perder los datos al cerrarlo, algo que quedó fuera de este trabajo porque la consigna pide que los datos vivan solo mientras el programa está en ejecución.